

SQL Query Architecture & Logic Engineering

☉ Status	SQL
☰ 선택	

SQL Query Architecture & Logic Engineering

1. 관계 대수와 집합론 (Foundations)

SQL은 본질적으로 관계 대수(Relational Algebra)의 상위 언어입니다. 구조화된 쿼리는 다음의 수학적 연산을 논리적으로 배치하는 설계도와 같습니다.

- **Selection (σ):** 특정 조건에 맞는 튜플을 선택 (WHERE)
- **Projection (π):** 필요한 속성(Attribute)만 추출 (SELECT)
- **Join ($\bowtie$):** 공통 속성을 기반으로 두 관계를 결합
- **Division ($\div$):** 모든 조건을 만족하는 집합을 탐색 (주로 복합 서브쿼리로 구현)

2. 논리적 쿼리 처리 순서 (Logical Query Processing)

Itzik Ben-Gan이 정의한 이 순서는 쿼리 작성 순서와 다르며, 이를 이해하지 못하면 논리적 오류(예: SELECT에서 정의한 Alias를 WHERE에서 사용 불가)를 해결할 수 없습니다.

순서	구문 (Clause)	핵심 개념 및 주의사항
1	FROM & JOIN	데이터의 원천 집합 정의. 가장 큰 연산 비용 발생.
2	WHERE	Predicate Pushdown: 데이터 셋을 조기 필터링하여 후속 연산 부하 감소.
3	GROUP BY	원소들을 그룹(Equivalence Class)으로 묶음. 집계 함수의 기점.
4	HAVING	집계된 그룹 집합에 대한 필터링.
5	SELECT	Projection: 최종 결과에 필요한 컬럼만 선택. Window Function 수행.

6	DISTINCT	중복 제거. 집합론적 유일성 확보.
7	ORDER BY	결과 정렬. 관계형 모델은 순서가 없으므로 최종 출력 단계에서만 수행.
8	LIMIT / TOP	최종 출력 행수 제한.

3. 현대적 구조화 패턴: Modular SQL (dbt Style)

최근 북미 데이터 엔지니어링 생태계(dbt, Snowflake)에서 강조하는 '**모듈화 (Modularity)**' 원칙입니다. 거대한 쿼리를 하나의 문장으로 쓰지 않고 CTE를 사용하여 계층화합니다.

3.1. CTE Layering (계층적 설계)

1. **Staging Layer (import)**: 원천 테이블에서 필요한 컬럼만 추출하고 데이터 타입을 캐스팅합니다.
2. **Intermediate Layer (logic)**: 비즈니스 로직(CASE 문, 날짜 계산 등)을 처리하여 파생 변수를 생성합니다.
3. **Core Layer (join/agg)**: 정제된 데이터들을 결합하고 집계를 수행합니다.
4. **Final Layer (output)**: 최종 사용자에게 보여줄 형태로 정렬 및 정리합니다.

3.2. The Rule of One Row

쿼리의 각 단계에서 "이 결과 집합의 한 행이 무엇을 의미하는가?"를 정의해야 합니다. Join 시 입도(Granularity)가 다르면 데이터 왜곡(Fan-out)이 발생하여 분석 결과가 수십 배 부풀려질 수 있습니다.

4. 성능 지향적 구조화: SARGability & Optimization

외국 기업의 시니어 데이터 엔지니어들이 가장 중요하게 여기는 **SARGable(Search Argumentable)** 쿼리 설계 기법입니다.

- **인덱스 친화적 필터링**: WHERE 절의 컬럼을 가공하지 마십시오.
 - *Bad*: `WHERE DATE_FORMAT(created_at, '%Y') = '2026'` (인덱스 파괴)
 - *Good*: `WHERE created_at >= '2026-01-01' AND created_at < '2027-01-01'`
- **Subquery vs JOIN vs EXISTS**:
 - 데이터 존재 여부 확인 시에는 `IN` 이나 `JOIN` 보다 `EXISTS` 가 효율적일 때가 많습니다 (Semi-join 최적화).

- **UNION ALL의 우선 사용:** 중복 제거가 명확히 필요하지 않다면 `UNION` 대신 연산 비용이 훨씬 적은 `UNION ALL` 을 사용하십시오.

5. 고급 로직 설계 (Advanced Analytical Patterns)

실무에서 복잡한 질문에 답하기 위해 반드시 포함해야 할 구조적 기술들입니다.

5.1. Window Functions (Analytical Engineering)

`GROUP BY` 의 한계를 넘어, 행의 정체성을 유지하면서 통계량을 산출합니다.

- **Frame Specification:** `ROWS BETWEEN 6 PRECEDING AND CURRENT ROW` 를 활용한 이동 평균 산출.
- **Ranking:** `ROW_NUMBER()` , `RANK()` , `DENSE_RANK()` 의 논리적 차이 활용.

5.2. Hierarchical & Recursive Queries

조직도, 카테고리 트리 등 계층 데이터를 처리하기 위한 `WITH RECURSIVE` 구조화 방법론입니다.

5.3. Conditional Aggregation

`CASE WHEN` 을 집계 함수 안에 배치하여 하나의 쿼리 안에서 여러 피벗(Pivot) 결과물을 동시에 산출하는 기법입니다.

6. 쿼리 검증 프레임워크 (QA & Validation)

구조화된 쿼리가 '정답'임을 증명하기 위한 4단계 검증 프로세스입니다.

1. **Cardinality Check:** Join 전후의 행 수가 비즈니스 관계(1:1, 1:N)에 부합하는가?
2. **NULL Sensitivity:** `COUNT(*)` 와 `COUNT(col)` 의 차이, `SUM` 의 NULL 처리 결과가 의도와 일치하는가?
3. **Boundary Analysis:** 시간 경계값(23:59:59), 최소/최대값 데이터가 포함되었는가?
4. **Cross-Validation:** 서로 다른 경로로 계산된 지표가 일치하는가? (예: 전체 매출 = 지역별 매출의 합)

7. 스타일 가이드 및 가독성 (Readability)

협업을 위한 글로벌 표준 SQL 스타일(SQLFluff 가이드라인 참고)입니다.

- **선행 쉼표(Leading Commas):** 컬럼 추가/삭제 시 가독성 확보.

- **명시적 별칭(Explicit Aliasing):** 모든 컬럼에 테이블 별칭(`t1.user_id`)을 부여하여 출처 명시.
- **의미 있는 CTE 명명:** `cte1` , `temp` 대신 `stg_users` , `int_order_summary` 등 직관적 명칭 사용.

8. 실무 비즈니스 분석 쿼리 설계 패턴 (Business Analytical Patterns)

앞서 다룬 논리적 구조화와 Window Function을 활용하여, 실제 비즈니스 도메인에서 가장 빈번하게 요구되는 핵심 분석 모델을 SQL로 설계하는 방법론입니다.

8.1. 코호트 리텐션 분석 (Cohort & Retention Analysis)

특정 기간에 유입된 사용자 그룹(Cohort)이 시간이 지남에 따라 얼마나 잔존(Retention)하는지 추적하는 쿼리 구조입니다.

- **Layer 1 (사용자 최초 획득일):** 각 사용자의 최초 활동일(Acquisition Date)을 정의합니다. `MIN()` 함수를 활용합니다.
- **Layer 2 (활동 이력 조인):** 최초 획득일 집합과 전체 활동 이력을 조인하여, 최초 획득일 대비 이후 활동일까지의 시간 차이(Time Delta)를 계산합니다. `DATEDIFF` 또는 `EXTRACT` 를 사용합니다.
- **Layer 3 (피벗 및 집계):** `COUNT(DISTINCT user_id)` 와 Conditional Aggregation(`CASE WHEN`)을 결합하여 가로 형태의 리텐션 테이블(D+1, D+7, D+30)을 완성합니다.

8.2. 퍼널 분석 (Funnel Analysis & Conversion Tracking)

사용자가 서비스에 진입하여 최종 목표(예: 구매)에 이르기까지의 전환율과 이탈률을 추적합니다.

- **엄격한 퍼널 (Strict Funnel):** 사용자가 A → B → C 단계를 순서대로, 연속적으로 밟았는지 확인합니다. `LEAD()` 윈도우 함수를 사용하여 다음 단계의 이벤트 시간과 이름을 현재 행으로 가져와 비교합니다.
- **느슨한 퍼널 (Loose Funnel):** 순서와 무관하게 특정 기간 내에 A, B, C를 모두 수행했는지 확인합니다. 각 단계별로 CTE를 만들고 `LEFT JOIN` 으로 결합하여 전환 여부를 측정합니다.

8.3. 세션화 (Sessionization : Gaps and Islands Problem)

웹/앱 로그 데이터에서 연속된 행동을 하나의 '세션(Session)'으로 묶는 고급 쿼리 기법입니다. 보통 30분의 유휴 시간이 발생하면 새로운 세션으로 간주합니다.

1. **지연 시간 계산 (Lag Calculation):** `LAG(event_time) OVER(PARTITION BY user_id ORDER BY event_time)` 을 사용하여 이전 이벤트와의 시간 차이를 구합니다.
2. **세션 경계 식별 (New Session Flag):** 시간 차이가 30분을 초과하면 1, 아니면 0을 부여하는 플래그 컬럼을 생성합니다.
3. **세션 ID 부여 (Cumulative Sum):** 플래그 컬럼에 대해 누적 합계(`SUM() OVER(PARTITION BY user_id ORDER BY event_time)`)를 구하면, 고유한 세션 ID가 형성됩니다.

9. 비즈니스 KPI 및 데이터 마트 설계 로직 (KPI & Data Mart Engineering)

단순한 Ad-hoc 추출을 넘어, 전사적 지표를 모니터링하기 위한 집계 테이블(Aggregate Table)의 구조를 잡는 원칙입니다.

9.1. 다차원 매출 KPI 테이블 구조화

브랜드별, 카테고리별, 지역별 매출 추이를 다각도로 분석하기 위한 OBT(One Big Table) 설계 방법론입니다.

- **선행 지표와 후행 지표의 분리:** 결제 완료 건수(선행)와 최종 정산 금액/환불 금액(후행)을 동일한 타임라인에서 볼 수 있도록 처리해야 합니다.
- **차원(Dimension) 확장:** 트랜잭션 팩트(Fact) 테이블에 브랜드 정보, 상품 마스터 정보를 조인하여 하나의 넓은 테이블로 만듭니다. 이때 Fan-out(데이터 뿔튀기)이 발생하지 않도록 1:N 관계 조인에 각별히 주의해야 합니다.
- **집계 입도(Granularity) 표준화:** 보통 `YYYY-MM-DD` 일별 입도로 1차 집계(Daily Aggregation)를 수행하여, BI 대시보드 연결 시 쿼리 성능을 최적화합니다.

9.2. 스냅샷 데이터와 상태 변경 추적 (SCD Type 2)

구독 서비스의 등급 변경, 상품 가격의 변동 등 이력이 누적되는 비즈니스 로직을 처리하는 패턴입니다.

- **유효 기간 설계:** 데이터 모델링 시 `valid_from` 과 `valid_to` 컬럼을 둡니다. 현재 활성화된 데이터는 `valid_to` 가 `9999-12-31` 이거나 `NULL` 로 설정됩니다.
- **시점 기준 조인 (Point-in-Time Join):** 특정 시점(예: 3개월 전)의 사용자 등급을 기준으로 당시의 매출을 계산하려면, 트랜잭션 발생일이 `valid_from` 과 `valid_to` 사이에 있는(`BETWEEN`) 조건으로 조인해야 정확한 과거 상태를 복원할 수 있습니다.

10. 비즈니스 요구사항의 SQL 번역 프로세스 (Translation Framework)

현업의 막연한 요구사항을 오차 없는 SQL 코드로 변환하는 4단계 사고 프로세스입니다.

단계	현업의 요구사항 (예시)	데이터 엔지니어/분석가의 해석 및 SQL 로직
1. 용어 명확화	"최근 한 달간의 진성 유저 매출"	- '최근 한 달': <code>CURRENT_DATE</code> 기준 D-30 인가, 전월(Last Month) 1일~말일인가? - '진성 유저': 누적 구매 3회 이상인가, 체류 시간 기준인가? - '매출': 결제액 기준인가, 환불 제외 순매출(Net Revenue)인가?
2. 입도 (Granularity) 설정	-	최종 결과물의 한 줄(Row)은 무엇을 의미하는가? (예: 유저 1명당 1행, 브랜드-일자별 1행)
3. 예외 및 엣지 케이스	"탈퇴한 유저도 포함하나요?"	- Null 값 처리 방법 (<code>COALESCE</code> 사용 여부) - 데이터가 아예 없는 날짜의 노출 여부 (Date Dimension 테이블과 <code>RIGHT JOIN</code> 필요성)
4. 쿼리 아키텍처 매핑	-	1. CTE 1: 진성 유저 ID 추출 (필터링) 2. CTE 2: 환불 제외 순매출 계산 3. Final: 두 CTE를 <code>INNER JOIN</code> 후 집계